

Rapport d'Analyse

1. Objectif de l'Expérience

L'objectif de ce benchmark est de mesurer et de comparer le temps de traitement de 809 images en niveaux de gris entre une approche séquentielle classique sur CPU (via OpenCV) et une approche sur GPU (via CuPy / CUDA 12.x). Deux algorithmes fondamentaux de la vision par ordinateur ont été testés :

Le Flou Gaussien (réduction du bruit par produit de convolution).

Le Filtre de Sobel (détection de contours par calcul des gradients horizontaux et verticaux).

2. Résultats Expérimentaux (Données Brutes)

Les mesures chronométriques exactes relevées lors de l'exécution du notebook sont les suivantes :

Gaussian Blur

CPU : 0.02309274673461914

GPU : 0.17374157905578613

Sobel

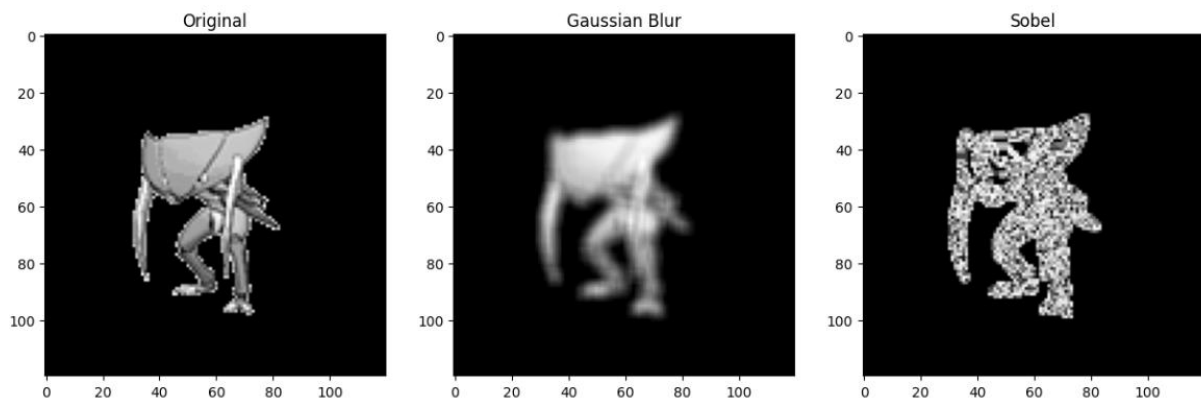
CPU : 0.09571027755737305

GPU : 0.573509693145752

Tableau Récapitulatif

Algorithme	Temps CPU (OpenCV)	Temps GPU (CuPy)	Performance Relative
Gaussian Blur	0.0231 s	0.1737 s	Le CPU est 7,52x plus rapide
Sobel Filter	0.0957 s	0.5735 s	Le CPU est 5,99x plus rapide

3. Visualisation des Traitements



4. Analyse Technique du Paradoxe CPU/GPU

Les résultats démontrent que le CPU est nettement plus performant que le GPU sur cette implémentation, ce qui peut sembler contre-intuitif. L'analyse du code source révèle l'origine exacte de ce comportement :

Le Goulot d'Étranglement : La boucle de transfert mémoire

Dans le script GPU actuel, les images sont traitées de manière isolée via une boucle for :

Python

```
for img in images:

    gpu_img = cp.asarray(img)

    sx = sobel(gpu_img, axis=0)
    sy = sobel(gpu_img, axis=1)

    magnitude = cp.sqrt(sx**2 + sy**2)
```

Pour chacune des 809 images, le système subit la latence d'accès du bus PCIe pour envoyer l'image au GPU, puis pour récupérer le résultat. Le coût temporel de ces 1618 transferts de données est largement supérieur au temps requis pour effectuer les calculs mathématiques. Le GPU passe son temps à attendre les données.

L'efficacité d'OpenCV sur CPU

À l'inverse, OpenCV exécute ses fonctions directement sur la mémoire RAM où les images sont déjà chargées. De plus, OpenCV intègre des optimisations SIMD (Single Instruction Multiple Data) au niveau du processeur, ce qui le rend extrêmement véloce pour le traitement séquentiel d'images de taille standard.

